Corolar

Există și este unică $Var : Form \to \mathcal{P}(V)$ astfel încât:

- pentru orice $v \in V$, $Var(v) = \{v\}$;
- pentru orice $\varphi \in Form$, $Var(\neg\varphi) = Var(\varphi)$;
- pentru orice $\varphi, \psi \in Form$, $Var(\varphi \to \psi) = Var(\varphi) \cup Var(\psi)$.

De exemplu,
$Var(\neg v_0 \to v_1) = Var(\neg v_0) \cup Var(v_1) =$
$Var(v_0) \cup Var(v_1) = \{v_0\} \cup \{v_1\} = \{v_0, v_1\}$.

Regulile de deducție big-step

$$\frac{}{(\textbf{skip}, \sigma) \Downarrow \sigma}$$

$$\frac{}{(X := a, \sigma) \Downarrow \sigma_{X \mapsto e_\sigma(a)}} \qquad \frac{(c_0, \sigma) \Downarrow \sigma'' \quad (c_1, \sigma'') \Downarrow \sigma'}{(c_0; c_1, \sigma) \Downarrow \sigma'}$$

$$\frac{e_\sigma(b) = 1 \quad (c_0, \sigma) \Downarrow \sigma'}{(\textbf{if } b \textbf{ then } c_0 \textbf{ else } c_1, \sigma) \Downarrow \sigma'} \qquad \frac{e_\sigma(b) = 0 \quad (c_1, \sigma) \Downarrow \sigma'}{(\textbf{if } b \textbf{ then } c_0 \textbf{ else } c_1, \sigma) \Downarrow \sigma'}$$

$$\frac{e_\sigma(b) = 0}{(\textbf{while } b \textbf{ do } c, \sigma) \Downarrow \sigma}$$

$$\frac{e_\sigma(b) = 1 \quad (c, \sigma) \Downarrow \sigma'' \quad (\textbf{while } b \textbf{ do } c, \sigma'') \Downarrow \sigma'}{(\textbf{while } b \textbf{ do } c, \sigma) \Downarrow \sigma'}$$

Regulile de deducție small-step

$$\frac{}{(X := a, \sigma) \to (\textbf{skip}, \sigma_{X \mapsto e_\sigma(a)})}$$

$$\frac{}{(\textbf{skip}; c_1, \sigma) \to (c_1, \sigma)} \qquad \frac{(c_0, \sigma) \to (c_0', \sigma')}{(c_0; c_1, \sigma) \to (c_0'; c_1, \sigma')}$$

$$\frac{e_\sigma(b) = 1}{(\textbf{if } b \textbf{ then } c_0 \textbf{ else } c_1, \sigma) \to (c_0, \sigma)}$$

$$\frac{e_\sigma(b) = 0}{(\textbf{if } b \textbf{ then } c_0 \textbf{ else } c_1, \sigma) \to (c_1, \sigma)}$$

$$\frac{}{(\textbf{while } b \textbf{ do } c, \sigma) \to (\textbf{if } b \textbf{ then } (c; (\textbf{while } b \textbf{ do } c)) \textbf{ else skip}, \sigma)}$$

Echivalența între semantici

Teoremă

Fie c o instrucțiune și σ, $\sigma' \in \Sigma$. Atunci

$$(c, \sigma) \Downarrow \sigma' \text{ dacă și numai dacă } (c, \sigma) \to^* (\textbf{skip}, \sigma').$$

Demonstrație

Demonstrăm implicația „$\Rightarrow$”. Facem inducție după regulile pentru relația $\Downarrow$.

Tratăm cazul regulii $(\textbf{skip}, \sigma) \Downarrow \sigma$. Atunci, clar, $(\textbf{skip}, \sigma) \to^* (\textbf{skip}, \sigma)$. Cazul regulii pentru atribuire este similar.

La regula pentru secvențiere, deducem, din ipoteza de inducție, $(c_0, \sigma) \to^* (\textbf{skip}, \sigma'')$ și $(c_1, \sigma'') \to^* (\textbf{skip}, \sigma')$, de unde scoatem

$$(c_0; c_1, \sigma) \to^* (\textbf{skip}; c_1, \sigma'') \to (c_1, \sigma'') \to^* (\textbf{skip}, \sigma').$$

La prima regulă pentru **if** (cea de-a doua se tratează similar), deducem, din ipoteza de inducție, $(c_0, \sigma) \to^* (\textbf{skip}, \sigma')$, de unde scoatem

$$(\textbf{if } b \textbf{ then } c_0 \textbf{ else } c_1, \sigma) \to (c_0, \sigma) \to^* (\textbf{skip}, \sigma').$$

Notăm $w := \textbf{while } b \textbf{ do } c$. La prima regulă pentru **while**, unde $e_\sigma(b) = 0$, scoatem

$$(w, \sigma) \to (\textbf{if } b \textbf{ then } (c; w) \textbf{ else skip}, \sigma) \to (\textbf{skip}, \sigma).$$

La a doua, știm $(c, \sigma) \to^* (\textbf{skip}, \sigma'')$ și $(w, \sigma'') \to^* (\textbf{skip}, \sigma')$ și scoatem

$$\begin{aligned}(w, \sigma) &\to (\textbf{if } b \textbf{ then } (c; w) \textbf{ else skip}, \sigma) \\ &\to (c; w, \sigma) \to^* (\textbf{skip}; w, \sigma'') \to (w, \sigma'') \to^* (\textbf{skip}, \sigma').\end{aligned}$$

Demonstrăm implicația „$\Leftarrow$”. Acum vom face inducție structurală după c.

Pentru instrucțiunea **skip**, dacă avem $(\textbf{skip}, \sigma) \to^* (\textbf{skip}, \sigma')$, atunci neapărat $\sigma = \sigma'$ și, clar, avem $(\textbf{skip}, \sigma) \Downarrow \sigma$. Cazul instrucțiunii de atribuire este similar.

Dacă știm $(c_0; c_1, \sigma) \to^* (\textbf{skip}, \sigma')$, clar trebuie să avem

$$(c_0; c_1, \sigma) \to^* (\textbf{skip}; c_1, \sigma'') \to (c_1, \sigma'') \to^* (\textbf{skip}, \sigma')$$

și în particular $(c_0, \sigma) \to^* (\textbf{skip}, \sigma'')$. De aici scoatem $(c_0, \sigma) \Downarrow \sigma''$ și $(c_1, \sigma'') \Downarrow \sigma'$, deci $(c_0; c_1, \sigma) \Downarrow \sigma'$.

Algebra IMP

Cum formalizăm acum limbajul IMP?

Vom pune $S := \{a, b, i\}$ (sorturi pentru expresii aritmetice, booleene și instrucțiuni), iar în E toate operațiile pe care le-am definit, pe care nu le vom enumera. Vom spune doar care este aritatea fiecărei operații care produce elemente de sort i:

- $r(\textbf{skip}) = (\lambda, i)$;
- pentru orice $X \in L$, $r(X :=) = (a, i)$;
- $r(;) = (ii, i)$;
- $r(\textbf{if}) = (bii, i)$;
- $r(\textbf{while}) = (bi, i)$.

Expresiile aritmetice, booleene și instrucțiunile, așa cum le-am definit, vor forma algebra inițială peste această signatură, algebră pe care o vom nota cu IMP.

Semantica este un morfism

Reamintim modul cum era definită semantica în logica propozițională: pentru orice $e : V \to \{0, 1\}$ există și este unică $e^+ : \mathit{Form} \to \{0, 1\}$ astfel încât:

- pentru orice $v \in V$, $e^+(v) = e(v)$;
- pentru orice $\varphi \in \mathit{Form}$, $e^+(\neg\varphi) = \neg e^+(\varphi)$;
- pentru orice $\varphi, \psi \in \mathit{Form}$, $e^+(\varphi \to \psi) = e^+(\varphi) \to e^+(\psi)$.

Dacă notăm, pentru orice φ și e, $[\![\varphi]\!](e) := e^+(\varphi)$, definind astfel o funcție $[\![\cdot]\!] : \mathit{Form} \to \{0,1\}^{\{0,1\}^V}$, și definind corespunzător operații $\tilde{\neg}$, $\tilde{\to}$, avem că:

- pentru orice $v \in V$, $[\![v]\!] = e \mapsto e(v)$;
- pentru orice $\varphi \in \mathit{Form}$, $[\![\neg\varphi]\!] = \tilde{\neg}[\![\varphi]\!]$;
- pentru orice $\varphi, \psi \in \mathit{Form}$, $[\![\varphi \to \psi]\!] = [\![\varphi]\!] \tilde{\to} [\![\psi]\!]$.

Vedem că semantica „scoate operații în afară", deci are trăsături de morfism. Ne putem întreba: se poate formaliza această idee, definind anumite structuri pentru care funcțiile chiar devin morfisme? putem face aceasta și pentru semantica denotațională a lui IMP pe care tocmai am definit-o?

Regulile de deducție Hoare

$$\frac{}{\{A\}\textbf{skip}\{A\}} \qquad \frac{}{\{B[X := a]\}X := a\{B\}}$$

$$\frac{\{A\}c_0\{C\} \quad \{C\}c_1\{B\}}{\{A\}c_0; c_1\{B\}} \qquad \frac{\{A \wedge b\}c_0\{B\} \quad \{A \wedge \neg b\}c_1\{B\}}{\{A\}\textbf{if } b \textbf{ then } c_0 \textbf{ else } c_1\{B\}}$$

$$\frac{\{A \wedge b\}c\{A\}}{\{A\}\textbf{while } b \textbf{ do } c\{A \wedge \neg b\}} \qquad \frac{\models A \to A' \quad \{A'\}c\{B'\} \quad \models B' \to B}{\{A\}c\{B\}}$$

Notăm faptul că o aserțiune $\{A\}c\{B\}$ poate fi dedusă prin aceste reguli cu $\vdash \{A\}c\{B\}$. Observăm și că ultima regulă, numită **regula consecinței**, este de fapt o **pseudo-regulă**, având în vedere că nu este decidabil dacă o aplicare a ei este corectă (de ce?).

Definirea celei mai slabe precondiții

Vom avea:

$$\begin{aligned} wp(\textbf{skip}, B) &:= B \\ wp(X := a, B) &:= B[X := a] \\ wp(c_0; c_1, B) &:= wp(c_0, wp(c_1, B)) \\ wp(\textbf{if } b \textbf{ then } c_0 \textbf{ else } c_1, B) &:= (b \wedge wp(c_0, B)) \vee (\neg b \wedge wp(c_1, B)). \end{aligned}$$

Definiția pentru instrucțiunea **while** va fi destul de migăloasă și, de aceea, doar vom schița ideile care stau la baza ei. Vom nota în continuare, ca de obicei, $w := \textbf{while } b \textbf{ do } c$. Scopul este de a defini $wp(w, B)$.

Teorema de completitudine

Lemă

Fie c, B cu $\vdash \{wp(c, B)\}c\{B\}$. Fie A cu $\models \{A\}c\{B\}$. Atunci $\vdash \{A\}c\{B\}$.

Demonstrație

Din regula consecinței, e suficient să arătăm $\models A \to wp(c, B)$. Fie I. Cum $\models^I \{A\}c\{B\}$, avem $A^I \subseteq ws^I(c, B) = wp(c, B)^I$, deci $\models^I A \to wp(c, B)$.

Teorema de completitudine

Pentru orice A, B, c cu $\models \{A\}c\{B\}$, avem $\vdash \{A\}c\{B\}$.

Demonstrație

Demonstrăm prin inducție structurală după c. Din lema anterioară, rezultă că este suficient, la fiecare pas, să arătăm că, pentru orice B, $\vdash \{wp(c, B)\}c\{B\}$.

Algoritm de unificare

Vom descrie acum un algoritm de unificare. El are ca intrare o mulțime de ecuații $\mathcal{E}$, iar la ieșire, un cgu dat de o mulțime de ecuații ca în propoziția precedentă, sau „eșec", dacă un cgu nu există. Algoritmul trece printr-un ciclu care, la fiecare pas, verifică dacă există vreo ecuație în mulțime căruia i se poate aplica vreuna dintre următoarele operații – în caz contrar, el se oprește:

- o ecuație de forma $f(s_1, \ldots, s_n) = f(t_1, \ldots, t_n)$ este înlocuită de ecuațiile $s_1 = t_1, \ldots, s_n = t_n$;
- o ecuație de forma $f(s_1, \ldots, s_n) = g(t_1, \ldots, t_m)$, unde $f \neq g$, produce „eșec";
- o ecuație de forma $x = x$, cu $x \in V$, este eliminată;
- o ecuație de forma $t = x$ cu $x \in V$ și $t \notin V$ este înlocuită cu $x = t$;
- o ecuație de forma $x = t$ cu $x \in V$, $t \neq x$ și $x \in Var(t)$ produce „eșec";
- o ecuație de forma $x = t$ cu $x \in V$, $t \neq x$, $x \notin Var(t)$, dar cu x apărând și în alte ecuații din mulțime, face ca toate celelalte sale apariții să fie substituite cu t.

Lambda-calcul fără tipuri

Fixăm o mulțime numărabilă de variabile $V = \{x_n \mid n \in \mathbb{N}\}$.

Un λ-**termen (fără tipuri)** are exact una dintre următoarele forme:

- o **variabilă** $x \in V$;
- o **(λ)-abstracțiune**: dacă $x \in V$ și M este un λ-termen, atunci $\lambda x.M$ este un λ-termen;
- o **aplicație**: dacă M și N sunt λ-termeni, atunci MN este un λ-termen.

Cum arată principiile de inducție/recursie corespunzătoare?

Variabile și variabile libere

Putem defini recursiv mulțimea variabilelor unui termen prin clauzele:

- $Var(v) = \{v\}$;
- $Var(\lambda x.M) = Var(M) \cup \{x\}$;
- $Var(MN) = Var(M) \cup Var(N)$;

iar pe cea a variabilelor libere prin:

- $FV(v) = \{v\}$;
- $FV(\lambda x.M) = FV(M) \setminus \{x\}$;
- $FV(MN) = FV(M) \cup FV(N)$.

Definirea substituției

Pentru orice λ-termeni M, N și orice $x \in V$, vom defini termenul $M[x := N]$, reprezentând M în care x a fost înlocuit cu N. O vom face recursiv, în felul următor (unde x, $y \in V$ cu $x \neq y$, iar N, P, Q sunt λ-termeni):

- $x[x := N] := N$;
- $x[y := N] := x$;
- $(PQ)[x := N] := (P[x := N])(Q[x := N])$;
- $(\lambda x.P)[x := N] := \lambda x.P$;
- $(\lambda y.P)[x := N] := \lambda y.(P[x := N])$, dacă $y \notin FV(N)$;
- $(\lambda y.P)[x := N] := \lambda z.(P[y := z][x := N])$, dacă $y \in FV(N)$, unde z este variabila de indice minim care nu aparține lui $Var(\lambda x.(NP))$, caz care corespunde fenomenului prezentat mai devreme.

Alte operații pe numere

Pentru adunare, observăm că, la fel ca înainte, pentru orice n, $m \in \mathbb{N}$, intuitiv vorbind

$$C_{n+m}(f)(x) = f^{(n+m)}(x) = f^{(n)}(f^{(m)}(x)) = C_n(f)(C_m(f)(x)),$$

ceea ce ne conduce la definiția:

$$\textbf{add} := \lambda n.\lambda m.\lambda f.\lambda x.((nf)((mf)x)).$$

Similar, putem defini:

$$\textbf{mul} := \lambda n.\lambda m.\lambda f.(n(mf)).$$

$$\textbf{pow} := \lambda b.\lambda e.(eb).$$

$$\textbf{pred} := \lambda n.\lambda f.\lambda x.(((n(\lambda g.\lambda h.(h(gf))))(\lambda u.x))(\lambda u.u)).$$

Recursivitate

Cum definim funcții recursive? Spre exemplu, funcția factorial ar satisface intuitiv ecuația

$$\textbf{fact} = \lambda n.(((\textbf{iszero}\ n)C_1)((\textbf{mul}\ n)(\textbf{fact}(\textbf{pred}\ n)))),$$

sau, altfel spus,

$$\textbf{fact} = (\lambda f.(\lambda n.(((\textbf{iszero}\ n)C_1)((\textbf{mul}\ n)(f(\textbf{pred}\ n))))))\textbf{fact}.$$

Dacă am avea la dispoziție un **operator de punct fix**, adică un λ-termen Y astfel încât, pentru orice M, $YM \to_\beta^* M(YM)$, atunci am putea, ca în Laboratorul 3, să definim

$$\textbf{fact} := Y(\lambda f.(\lambda n.(((\textbf{iszero}\ n)C_1)((\textbf{mul}\ n)(f(\textbf{pred}\ n)))))).$$

Puncte fixe

Dacă punem $U := \lambda u.\lambda x.(x((uu)x))$, atunci $Y := UU$ este un asemenea operator de punct fix, având în vedere că, pentru orice M, avem

$$\begin{aligned} YM &= (UU)M \\ &\to_\beta ((\lambda x.(x((uu)x)))[u := U])M \\ &= (\lambda x.(x((UU)x)))M \\ &= (\lambda x.(x(Yx)))M \\ &\to_\beta (x(Yx))[x := M] \\ &= M(YM). \end{aligned}$$

Această soluție a fost dată de Alan Turing. Altă soluție, dată de Haskell Curry, este $Y := \lambda u.((\lambda x.u(xx))(\lambda x.u(xx)))$.

Reguli de deducție

Regulile de deducție vor fi următoarele:

$$\frac{}{\Gamma \cup \{x : \sigma\} \vdash x : \sigma}$$

$$\frac{\Gamma \cup \{x : \sigma\} \vdash M : \tau}{\Gamma \vdash \lambda x.M : \sigma \to \tau} \qquad \frac{\Gamma \vdash M : \sigma \to \tau \quad \Gamma \vdash N : \sigma}{\Gamma \vdash MN : \tau}$$

Încercați să deduceți tipul termenului „de mai devreme” $(\lambda z.(\lambda u.z))(yx)$ folosind aceste reguli. Acest exemplu ne arată că, într-un anume fel (în ce fel? detaliați!), cele două moduri de a gândi tipurile sunt echivalente.

Algoritmul

Definim, pentru orice termen M, contextul

$$\Gamma_M := \{x : X \mid x \in FV(M)\}.$$

De asemenea, pentru orice termen M, construim termenul adnotat M', unde toate λx-urile devin $\lambda x : X$.

În continuare, vom defini o funcție c care primește ca argumente: un termen adnotat, un context și o variabilă de tip și care returnează o mulțime de ecuații peste signatura de ordinul I care conține doar simbolul săgeată (vezi discuția de la început).

Finalizare

Definim:

$$c(x, \Gamma \cup \{x : \tau\}, Z) := \{\tau = Z\}$$
$$c(\lambda x : \sigma.M, \Gamma, Z) := c(M, \Gamma \cup \{x : \sigma\}, W) \cup \{Z = \sigma \to W\}$$
$$c(MN, \Gamma, Z) := c(M, \Gamma, W_1) \cup c(N, \Gamma, W_2) \cup \{W_1 = W_2 \to Z\}$$

Algoritmul va face apelul $c(M', \Gamma_M, Z)$ și va obține o mulțime de ecuații. Pentru ea, se caută un cgu θ. În caz că nu există, se returnează „eșec". În caz de succes, rezultatul algoritmului va fi $\widetilde{\theta}(\Gamma_M) \vdash M : \widetilde{\theta}(Z)$ (unde $\widetilde{\theta}$ are semnificația firească).

Feluri de limbaje funcționale

Până acum am studiat două modele matematice ale programării funcționale:

- λ-calculul fără tipuri
  - un model de calcul complet, având și parțialitate
  - un limbaj de programare modelat oarecum pe baza lui: Lisp
- λ-calcul cu tipuri (simple sau mai complicate)
  - modelează o subclasă strictă a clasei funcțiilor calculabile totale
  - un limbaj de programare oarecum în acest stil: Agda

Pentru a modela limbajul Haskell, avem nevoie de un model de calcul complet care să includă și un sistem de tipuri. În continuare, vom prezenta un exemplu de așa ceva, anume PCF (Programming Computable Functions), introdus de Scott și Plotkin în anii '70.

Tipuri și termeni

Tipurile vor fi următoarele:

- un singur tip de bază, $\mathbb{N}$ (reprezentând numerele naturale);
- pentru orice tipuri ρ, τ, avem un tip $\rho \to \tau$ (reprezentând funcțiile parțiale „de la ρ la τ").

Vom defini termenii fără tipuri asociate, și apoi le vom aloca tipuri „à la Curry":

- avem variabile, λ-abstracțiuni și aplicații exact ca la λ-calculul fără tipuri;
- z este termen („zero");
- dacă M este termen, atunci $s(M)$ este termen („succesor");
- dacă M, N, P sunt termeni, iar w este variabilă, atunci $ifz(M, N, w, P)$ este termen;
- dacă M este termen, iar w este variabilă, atunci $fix(w, M)$ este termen.

Alocarea tipurilor

Acestea sunt regulile de deducție pentru alocarea de tipuri termenilor:

$$\frac{}{\Gamma \cup \{x : \sigma\} \vdash x : \sigma}$$

$$\frac{\Gamma \cup \{x : \sigma\} \vdash M : \tau}{\Gamma \vdash \lambda x.M : \sigma \to \tau} \qquad \frac{\Gamma \vdash M : \sigma \to \tau \quad \Gamma \vdash N : \sigma}{\Gamma \vdash MN : \tau}$$

$$\frac{}{\Gamma \vdash z : \mathbb{N}} \qquad \frac{\Gamma \vdash M : \mathbb{N}}{\Gamma \vdash s(M) : \mathbb{N}}$$

$$\frac{\Gamma \vdash M : \mathbb{N} \quad \Gamma \vdash N : \tau \quad \Gamma \cup \{w : \mathbb{N}\} \vdash P : \tau}{\Gamma \vdash ifz(M, N, w, P) : \tau} \qquad \frac{\Gamma \cup \{w : \tau\} \vdash M : \tau}{\Gamma \vdash fix(w, M) : \tau}$$

Valori

Următoarele reguli stabilesc „când ceva este valoare”:

$$\frac{}{val(z)} \qquad \frac{val(M)}{val(s(M))} \qquad \frac{}{val(\lambda x.M)}$$

Reguli de evaluare

Următoarele reguli stabilesc evaluarea termenilor în regim *eager*:

$$\frac{M \to M'}{s(M) \to s(M')} \qquad \frac{M \to M'}{ifz(M, N, w, P) \to ifz(M', N, w, P)}$$

$$\frac{}{ifz(z, N, w, P) \to N} \qquad \frac{val(s(M))}{ifz(s(M), N, w, P) \to P[w := M]}$$

$$\frac{M \to M'}{MN \to M'N} \qquad \frac{val(M) \quad N \to N'}{MN \to MN'}$$

$$\frac{val(N)}{(\lambda x.M)N \to M[x := N]} \qquad \frac{}{fix(w, M) \to M[w := fix(w, M)]}$$

Exemple de termeni

Operația de adunare este reprezentată de următorul termen PCF (de ce?):

$$fix(f, \lambda x.\lambda y.ifz(y, x, w, s((fx)w))).$$

Putem defini operația de „scădere cu punct", pentru orice x, $y \in \mathbb{N}$, prin:

$$x \dot{-} y = \begin{cases} x - y, & \text{dacă } x \geqslant y, \\ 0, & \text{altfel.} \end{cases}$$

Ea se poate reprezenta prin (de ce?):

$$fix(f, \lambda x.\lambda y.ifz(y, x, w, ifz(x, z, u, (fu)w))).$$

Notăm acest termen tot cu $\dot{-}$.

Un termen care reprezintă funcția de cel mai mare divizor comun, obținut prin algoritmul lui Euclid, este:

$$fix(f, \lambda x.\lambda y.ifz(x, y, w, ifz(y, x, u, ifz((\dot{-}y)x, f((\dot{-}x)y)y, v,$$

$$(f(x))(((\dot{-}y)x))))))).$$

În sfârșit, definim operatorul de *minimizare nemărginită*, μ, pentru orice funcție parțială f, punând $\mu(f)$ ca fiind, în caz că există (altfel, rămâne nedefinit), cel mai mic număr natural n cu proprietatea că $f(n) = 0$ și că f este definită pentru toate numerele mai mici decât n. Un termen care îl reprezintă pe μ este:

$$\lambda f.((fix(g, \lambda n.\lambda f.ifz(fn, n, w, (g(Sn))f))z)f).$$